GUÍA DE ESTUDIO PARA LA PRESENTACIÓN

Cómo funciona el programa que reconoce y nombra las fotos y los DPI

El archivo organizar_fotos.py explicado desde cero: Python, visión por computadora, cada función y cada ciclo

Para quién	Quien va a presentar el proyecto y no ha programado antes
Qué se explica	Qué hace el programa, con qué herramientas, cómo piensa cada función y cómo defenderlo ante preguntas
Archivo estudiado	organizar_fotos.py (versión entregada con el proyecto, 1,046 líneas)
Datos de ejemplo	Todos los nombres, números y coordenadas de este documento son inventados

Contenido

0. Primero, tu pregunta: ¿a dónde van las imágenes? 4

1. El programa en un minuto 5

2. Python mínimo para poder leer el archivo 7

2.1 Cómo está organizado el archivo . . . 7

2.2 Las piezas de sintaxis, con ejemplos del propio archivo . . . 8

2.3 Los ciclos más importantes del programa . . . 8

2.4 La “ficha” de cada imagen: el diccionario que viaja por todo el programa . . . 9

3. Visión por computadora desde cero 10

3.1 Una imagen es una tabla de números . . . 10

3.2 Recortar: mirar solo la zona que importa . . . 10

3.3 Pasar a gris y agrandar . . . 11

3.4 Umbralización: blanco o negro, sin grises . . . 11

3.5 Contraste: histograma, equalizeHist y CLAHE . . . 13

3.6 OCR: convertir una imagen de texto en texto . . . 13

3.7 Detección de rostros con cascadas de Haar . . . 14

3.8 Girar la imagen: saber si el DPI está derecho . . . 15

3.9 La zona de lectura mecánica y el dígito de control . . . 15

3.10 Expresiones regulares: buscar patrones en el texto . . . 16

4. Recorrido por el código, función por función 17

4.1 La configuración (líneas 60 a 108) . . . 17

4.2 Funciones para trabajar con nombres (líneas 120 a 207) . . . 17

4.3 ocr_overlay (líneas 217 a 281): leer la etiqueta . . . 18

4.4 proporcion_cara y proporcion_cara_laxa (líneas 303 a 353): medir la cara . . . 18

4.5 Lectura del DPI (líneas 361 a 523) . . . 18

4.6 analizar (líneas 529 a 564): el ciclo principal . . . 19

4.7 agrupar_personas (líneas 567 a 594): quién es quién . . . 19

4.8 asignar_roles (líneas 597 a 679): quién es cada foto . . . 20

4.9 emparejar_dpi (líneas 685 a 843): de quién es cada DPI . . . 20

4.10 Guardar los resultados (líneas 846 a 1042) . . . 21

5. Un ejemplo completo, paso a paso 22

6. Cuándo algo va a Revision_manual 22

7. Cómo explicarlo en la presentación 24

7.1 La explicación en tres minutos . . . 24

7.2 Preguntas que pueden hacerte 24
7.3 Lo que conviene decir y lo que no 25
7.4 Limitaciones y una mejora pendiente 25
8. Cómo seguir aprendiendo **26**
8.1 En qué orden leer el archivo 26
8.2 Experimentos seguros 26
8.3 Glosario 27
Anexo. Mapa de funciones del archivo **27**

0. Primero, tu pregunta: ¿a dónde van las imágenes?

Es una pregunta muy buena, y es justo la que hace un docente o un encargado de proteger datos personales: las fotos y los DPI de adultos mayores son información sensible. La respuesta corta es esta:

> **Las imágenes no van a ningún lado.**
> Todo el análisis ocurre dentro de tu propia computadora. El programa no se conecta a internet ni a ninguna página de inteligencia artificial. Las fotos se leen desde la carpeta, se analizan en la memoria de tu equipo y se copian a otra carpeta de tu mismo equipo.

¿Cómo sabemos que es así? Por cuatro razones que puedes explicar y comprobar:

- **El código no tiene ninguna pieza de red.** Al inicio del archivo (líneas 41 a 58) se listan las librerías que usa: manejo de archivos, texto, fechas, `cv2` (OpenCV), `numpy`, `pytesseract` y `openpyxl` (Excel). Ninguna sirve para conectarse a internet.
- **La "inteligencia" vive instalada en tu computadora.** Los modelos que detectan rostros son archivos `.xml` que vienen dentro del paquete OpenCV. El lector de texto es Tesseract, un programa que se instaló en `C:\Program Files\Tesseract-OCR`.
- **Internet solo se usó para instalar.** Los comandos `pip install` y el instalador de Tesseract descargaron las herramientas una sola vez. Después ya no hace falta conexión.
- **Se puede comprobar.** Desconecta el Wi-Fi, ejecuta el programa y funcionará igual.

¿Entonces es "inteligencia artificial"? Depende de qué parte:

Parte del programa	Qué técnica es	¿Dónde corre?
Detectar si hay una cara en la foto	Aprendizaje automático clásico (método de Viola y Jones, 2001). El modelo se entrenó con miles de ejemplos y quedó guardado en un archivo.	En tu computadora
Leer el texto de la etiqueta y del DPI (OCR)	Tesseract 5, que usa una red neuronal (tipo LSTM) para reconocer letras.	En tu computadora
Agrupar por persona, elegir la foto de medio cuerpo, emparejar el DPI, decidir qué va a revisión	**Reglas escritas por nosotros** (lógica de programación). No hay IA aquí.	En tu computadora

> **Dos aclaraciones para la presentación**
> 1) El programa **no usa IA generativa** ni servicios como ChatGPT. 2) El programa **no reconoce quién es una persona por su rostro**: solo detecta que hay una cara y mide qué tan grande se ve. Para saber de quién es una foto se apoya en el texto de la etiqueta, no en la cara.

Eso también contesta la duda de "las imágenes van a un lado durante el análisis": lo que se ve en pantalla (el avance `[12/90] WhatsApp Image…`) es el programa leyendo una por una las imágenes de tu carpeta.

1. El programa en un minuto

Piensa en lo que haría una persona de Registro y Archivo con una carpeta de fotos sin nombre: abriría cada foto, leería la etiqueta que trae impresa abajo, juntaría las fotos de la misma persona, buscaría la de medio cuerpo, ordenaría las demás, buscaría su DPI y pondría los nombres. El programa hace exactamente eso, en ese orden, y lo que no puede decidir con seguridad lo deja aparte para que lo revise una persona.

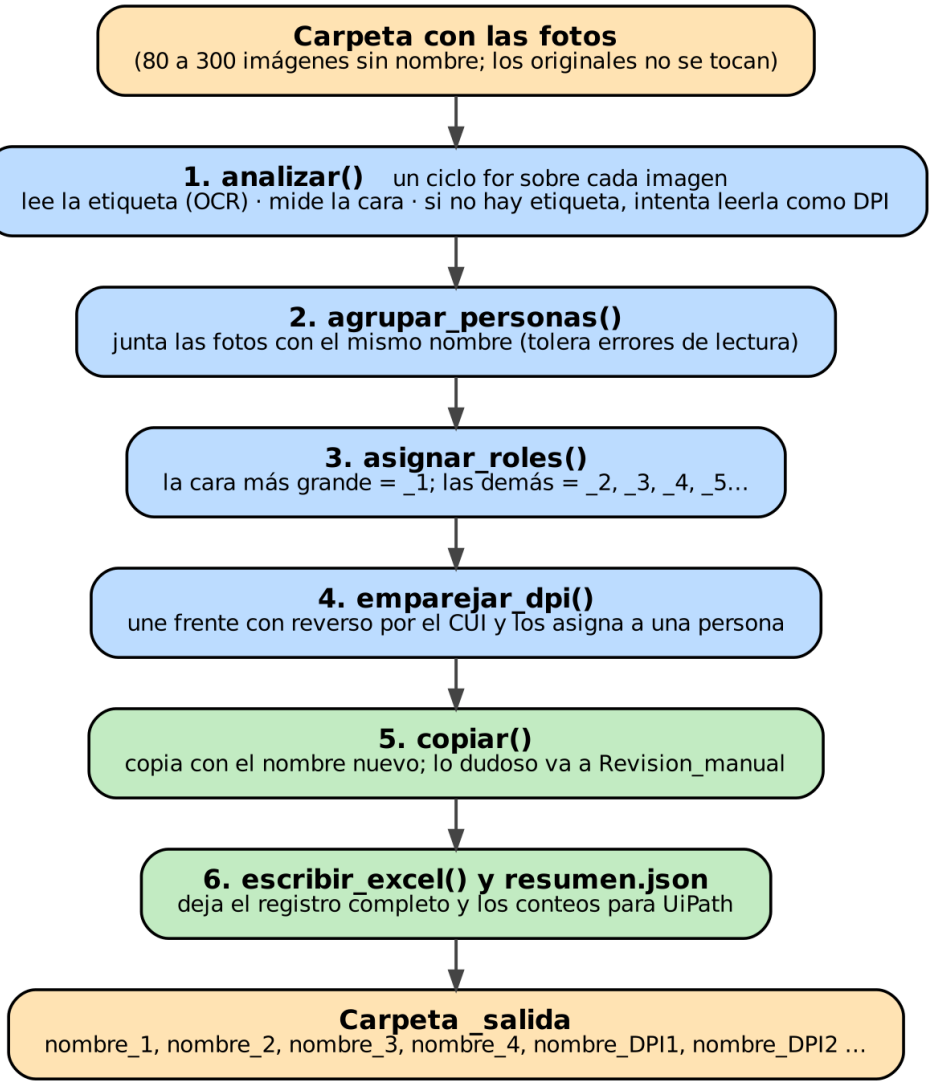


Figura 1. Las seis etapas del programa. Cada caja es una función del archivo.

Etapa	Pregunta que responde	Herramienta principal
`analizar()`	¿Qué dice la etiqueta de esta foto? ¿Qué tan grande es la cara? ¿Es un DPI?	OCR (Tesseract) y detector de rostros (OpenCV)
`agrupar_personas()`	¿Qué fotos son de la misma persona, aunque el nombre se haya leído con errores?	Comparación de textos (difflib)
`asignar_roles()`	¿Cuál es la de medio cuerpo y cuál es de la vivienda? ¿En qué orden van?	Tamaño de la cara y hora de captura

Etapa	Pregunta que responde	Herramienta principal
`emparejar_dpi()`	¿Qué frente va con qué reverso? ¿De quién es cada DPI?	CUI, dígito de control y nombres
`copiar()`	¿Dónde va cada archivo y con qué nombre?	Copias con el nombre exacto que exige la macro
`escribir_excel()`	¿Qué quedó registrado y qué hay que revisar?	Excel (openpyxl) y resumen.json

2. Python mínimo para poder leer el archivo

No necesitas saber programar para explicar el proyecto, pero sí reconocer las piezas. Un programa es una receta: una lista de pasos que la computadora sigue de arriba hacia abajo. Estas son las piezas que aparecen en el archivo.

2.1 Cómo está organizado el archivo

Líneas	Qué contiene	Analogía
1 a 40	El texto de presentación entre comillas triples (docstring): qué hace y cómo se usa	La portada de un manual
41 a 58	Las importaciones: las herramientas ya hechas que el programa pide prestadas	Sacar las herramientas de la caja
60 a 108	La configuración: números que se pueden ajustar y carga de los modelos de rostro	El tablero de perillas
111 a 524	Las funciones auxiliares: leer imágenes, limpiar nombres, OCR, rostros, lectura del DPI	Las herramientas del taller
529 a 977	Las funciones del proceso: analizar, agrupar, asignar roles, emparejar DPI, copiar, Excel	Las estaciones de la línea de producción
980 a 1046	La función `main()`: llama a las anteriores en orden y arma el resumen	El jefe de línea

2.2 Las piezas de sintaxis, con ejemplos del propio archivo

Lo que ves	Qué significa
`OVERLAY_ALTO = 0.14`	**Variable**: una caja con nombre que guarda un valor. Se escribe en MAYÚSCULAS cuando es una configuración que no cambia mientras corre el programa.
`EXTENSIONES = {".jpg", ".jpeg", ".png"}`	**Conjunto** (llaves): colección sin orden y sin repetidos. Sirve para preguntar rápido "¿está esto aquí?".
`UMBRALES = (90, 100, 110, 120, 130, 140)`	**Tupla** (paréntesis): lista que no se modifica. Aquí, los seis niveles de blanco y negro que se prueban.
`def leer_imagen(ruta):`	**Función**: un bloque con nombre que recibe datos (`ruta`) y devuelve un resultado con `return`. Se escribe una vez y se usa muchas.
`if ... elif ... else`	**Condición**: si pasa esto, haz aquello; si no, lo otro.
`for ruta in archivos:`	**Ciclo (bucle) for**: repite el bloque una vez por cada elemento de la lista.
`for i, ruta in enumerate(archivos, 1):`	El mismo ciclo, pero `enumerate` agrega un contador que empieza en 1, para poder mostrar `[12/90]`.
`continue / break`	`continue`: salta a la siguiente vuelta. `break`: sale del ciclo por completo.
`registro = dict(ruta=ruta, nombre=nombre)`	**Diccionario**: fichas de dato con etiqueta (`r["nombre"]` o `r.get("nombre")`). Cada foto es una "ficha".
`[r for r in registros if r.get('nombre')]`	**Lista por comprensión**: arma una lista nueva con solo los elementos que cumplen la condición.
`sorted(lista, key=lambda x: -x["votos"])`	Ordena la lista. `lambda` es una mini-función sin nombre que dice por qué campo ordenar; el signo menos invierte el orden.
`max(grupos, key=len)`	El elemento más grande según un criterio (aquí, el grupo con más lecturas).
`any(...) / sum(1 for ...)`	`any`: ¿se cumple en al menos uno? `sum(1 for ...)`: cuenta cuántos cumplen.
`alto, ancho = img.shape[:2]`	**Desempaquetar**: de los datos de la imagen toma los dos primeros y los guarda en dos variables.
`img[y0:y1, x0:x1]`	**Recorte (slicing)**: filas de `y0` a `y1` y columnas de `x0` a `x1`.
`try: ... except ...:`	**Manejo de errores**: intenta esto; si falla de esta forma, haz lo otro en vez de detenerte.
`f"[{i}/{total}] {ruta.name}"`	**Texto con huecos** (f-string): lo que va entre llaves se reemplaza por su valor.
`if __name__ == "__main__": main()`	Significa "si este archivo se ejecuta directamente, arranca por `main()`".

2.3 Los ciclos más importantes del programa

El programa es, en esencia, una cadena de ciclos. Estos son los cuatro que más conviene entender, porque los volverás a ver en la presentación.

Ciclo 1. Una vuelta por cada imagen (función `analizar`).

```
for i, ruta in enumerate(archivos, 1):      # archivos = todas las imágenes de la carpeta
    print(f"[{i}/{total}] {ruta.name}")       # lo que ves avanzar en pantalla
    img = leer_imagen(ruta)                   # abre la foto
    nombre, votos, hora, numero = ocr_overlay(img)   # lee la etiqueta
    ...                                       # decide si es foto, DPI o desconocida
```

Ciclo 2. Una vuelta por cada umbral (función `ocr_overlay`). Se explica en la sección 3.4: la misma etiqueta se lee seis veces, con seis niveles distintos de blanco y negro.

```
for u in UMBRALES:                          # u vale 90, luego 100, 110, 120, 130, 140
    _, bn = cv2.threshold(gris, u, 255, cv2.THRESH_BINARY)   # blanco y negro
    texto = pytesseract.image_to_string(bn, config="--psm 6")  # lee el texto
    for i, linea in enumerate(texto.splitlines()):             # ciclo dentro del ciclo
        ...                                 # busca "Nota:" y la hora en cada línea
```

Ciclo 3. El ciclo con "else" (funciones `ocr_overlay` y `agrupar_personas`). Es un rasgo particular de Python: el bloque `else` de un `for` se ejecuta **solo si el ciclo terminó sin hacer** `break`.

```
for n in nombres:
    for g in grupos:               # ¿ya existe un grupo parecido a este nombre?
        if mismo_nombre(n, g[0]):
            g.append(n)            # sí: lo agrego a ese grupo
            break                  # y dejo de buscar
    else:
        grupos.append([n])         # no encontré ninguno: creo un grupo nuevo
```

Ciclo 4. Ciclos para emparejar (función `emparejar_dpi`): se prueban todas las combinaciones reverso-frente y se unen primero las que tienen el CUI más parecido. Está explicado en la sección 4.9.

2.4 La "ficha" de cada imagen: el diccionario que viaja por todo el programa

Esta es la idea que más ayuda a leer el código: cada imagen se convierte en un diccionario (una ficha), y cada etapa **agrega campos nuevos** a la misma ficha. Nada se pierde y cada función sabe dónde seguir.

Etapa	Campos que agrega a la ficha	Ejemplo
`analizar`	`ruta, nombre, votos, hora, numero, cara, estado`	Maria Lopez Perez · 0.67 · 09:41 · 2468013570101 · 0.52
`agrupar_personas`	`persona, nombre_final`	marialopezperez
`asignar_roles`	`rol, criterio, nuevo, estado, detalle, hora_grupo`	medio_cuerpo · marialopezperez_1 · OK
`emparejar_dpi`	para los DPI: `persona, nuevo, estado, detalle`	marialopezperez_DPI1 · OK
`copiar`	`destino`	..._salida\marialopezperez_1.jpg

3. Visión por computadora desde cero

La visión por computadora es el conjunto de técnicas para que un programa extraiga información útil de una imagen. No es magia: parte de una idea muy simple (una imagen es una tabla de números) y le aplica operaciones sobre esos números. Estas son las que usa el programa.

3.1 Una imagen es una tabla de números

Una imagen digital es una cuadrícula de puntos llamados **píxeles**. En escala de grises, cada píxel es un número de 0 (negro) a 255 (blanco). En una imagen a color, cada píxel tiene tres números, uno por cada canal de color. OpenCV los guarda en el orden **azul, verde, rojo** (BGR), no en el RGB habitual. Una foto vertical de WhatsApp de 900 × 1598 píxeles tiene más de 1.4 millones de píxeles y cuatro millones de números.

Lo que ve una persona



Lo que ve el programa: una tabla de números
(0 = negro, 255 = blanco)

224	225	231	236	213	216	233	236
219	220	234	**28**	**24**	233	219	223
229	226	**20**	**48**	**69**	**36**	233	226
233	221	**29**	197	181	**25**	216	224
237	216	**27**	**28**	**40**	**23**	225	219
213	231	**19**	220	225	**30**	215	237
231	237	**20**	231	220	**31**	236	219
231	217	221	237	223	225	220	215

Figura 2. La letra A vista como persona (izquierda) y como el programa la ve (derecha). Los números bajos son los trazos oscuros.

En el código, `img.shape` devuelve las dimensiones: `(1598, 900, 3)` significa 1598 filas (alto), 900 columnas (ancho) y 3 canales. La función `leer_imagen` abre el archivo con `np.fromfile` y `cv2.imdecode` en lugar del `cv2.imread` habitual, porque este último falla con rutas que tienen tildes o espacios en Windows.

3.2 Recortar: mirar solo la zona que importa

Todas las fotos de Trabajo Social traen impresos, abajo a la izquierda, los datos de la visita: latitud, longitud, hora (`Tiempo`) y la línea `Nota` con el nombre. Leer la foto completa sería lento y confundiría al lector de texto con todo lo demás. Por eso el programa recorta solo el último 14 % del alto y el primer 55 % del ancho.

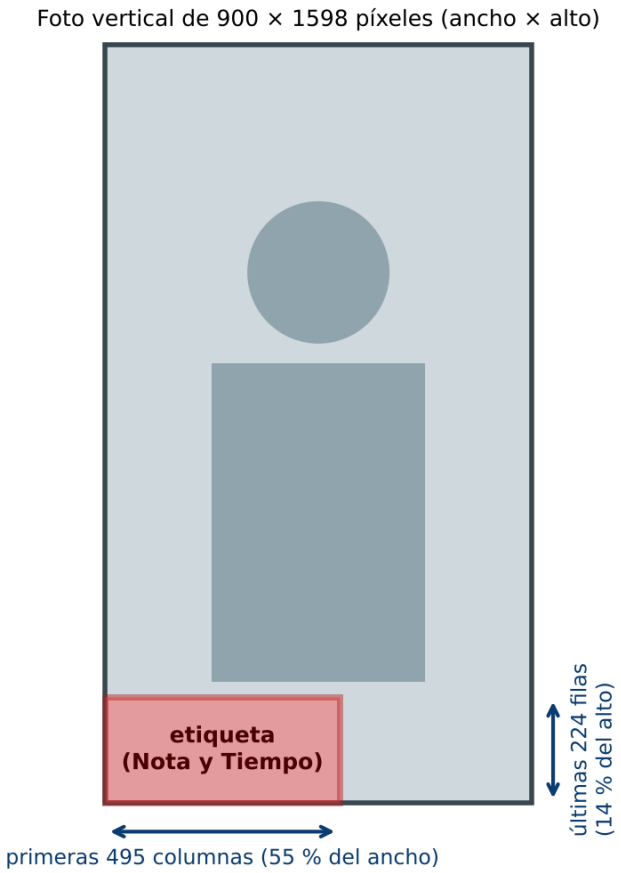


Figura 3. La zona de la etiqueta que se recorta (rojo). Las constantes son OVERLAY_ALTO = 0.14 y OVERLAY_ANCHO = 0.55.

```
alto, ancho = img.shape[:2]
recorte = img[int(alto * (1 - OVERLAY_ALTO)) : alto,  0 : int(ancho * OVERLAY_ANCHO)]
#             desde el 86 % de la altura hasta abajo,   desde la izquierda hasta el 55 % del ancho
```

3.3 Pasar a gris y agrandar

Para leer letras el color estorba. `cv2.cvtColor(recorte, cv2.COLOR_BGR2GRAY)` convierte cada píxel de tres números en uno solo. Después `cv2.resize(..., fx=3, fy=3, interpolation=cv2.INTER_CUBIC)` agranda el recorte tres veces: las letras de la etiqueta son pequeñas y a Tesseract le va mucho mejor con letras más grandes. La interpolación cúbica inventa los píxeles nuevos suavizando entre los vecinos para que los bordes no queden dentados.

3.4 Umbralización: blanco o negro, sin grises

La **umbralización** convierte cada píxel en blanco puro (255) o negro puro (0) según un número de corte, el umbral: si el brillo supera el umbral, queda blanco; si no, negro. En el caso de la etiqueta, que es texto oscuro sobre una caja clara, el resultado es texto negro sobre fondo blanco, que es lo que mejor lee Tesseract. El problema es que **no hay un umbral perfecto para todas las fotos**: con una foto oscura o con sol, el mismo número deja letras cortadas o fondo manchado.

La solución del programa es probar seis umbrales (`90, 100, 110, 120, 130, 140`) y **votar**: se lee el nombre con cada uno y gana el que más se repite. Es la misma idea de pedir la opinión a seis personas en vez de

confiar en una.

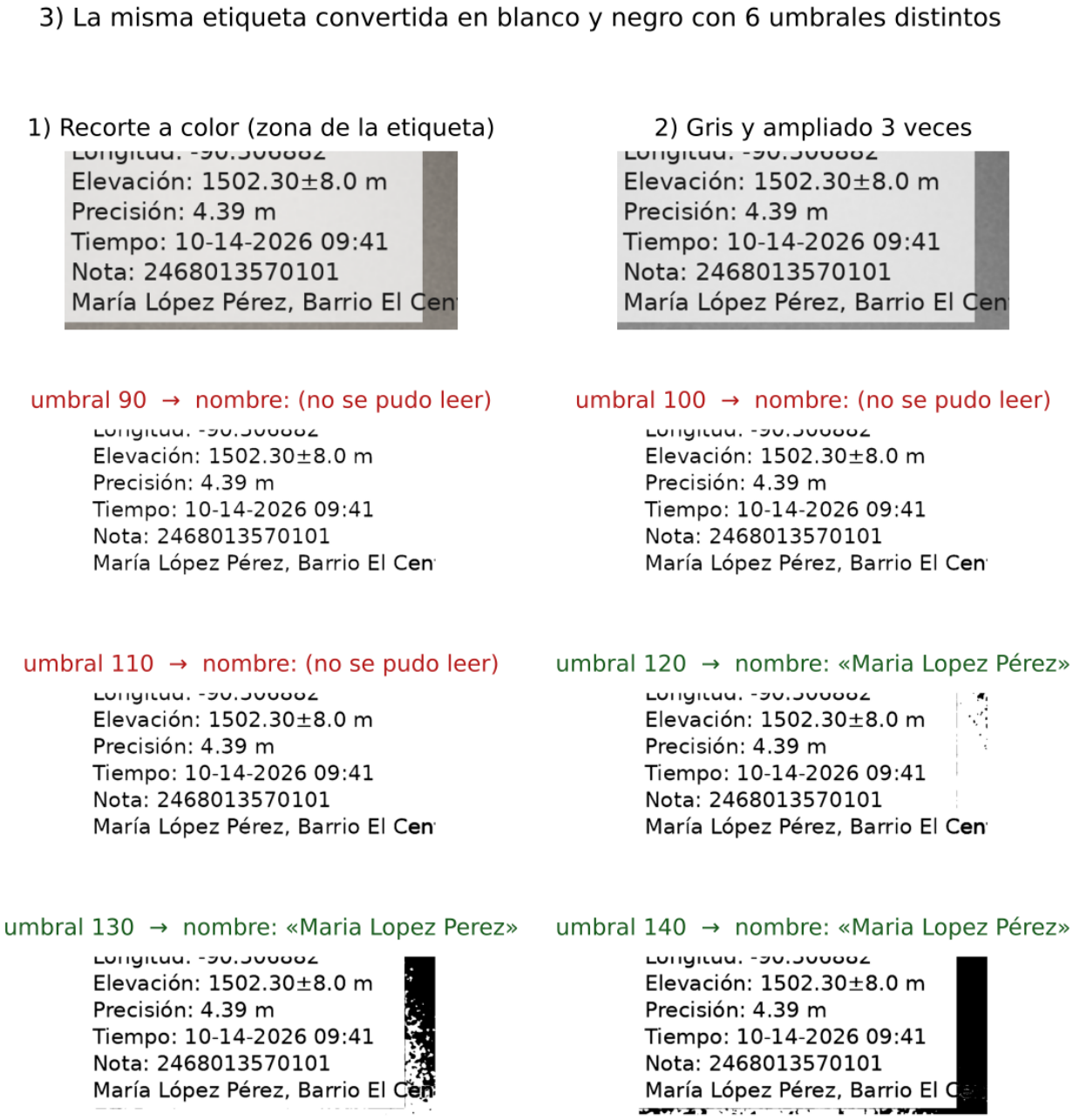


Figura 4. Resultado real del programa sobre una foto de ejemplo con datos inventados: con tres umbrales se leyó el nombre y con los otros tres no.

Fíjate en que en este ejemplo solo tres de los seis umbrales lograron leer el nombre, y aun así la votación entregó el resultado correcto. Para el programa, **votos = 3 / 6 = 0.5**: es la "confianza" que luego aparece en el registro de Excel. Si ninguna otra foto de la persona respalda el nombre y los votos son menores de `MIN_VOTOS = 0.34`, la foto se marca para revisión.

3.5 Contraste: histograma, equalizeHist y CLAHE

El **histograma** cuenta cuántos píxeles hay de cada brillo. Una foto con poco contraste tiene todos sus píxeles amontonados en una franja estrecha, y por eso los detalles se ven “lavados”. La **ecualización** estira esa franja para usar todo el rango de 0 a 255. `equalizeHist` lo hace con toda la imagen a la vez; `CLAHE` (siglas en inglés de ecualización adaptativa con límite de contraste) lo hace por cuadros de 8 × 8 zonas, y por eso no exagera el contraste en lugares ya claros.

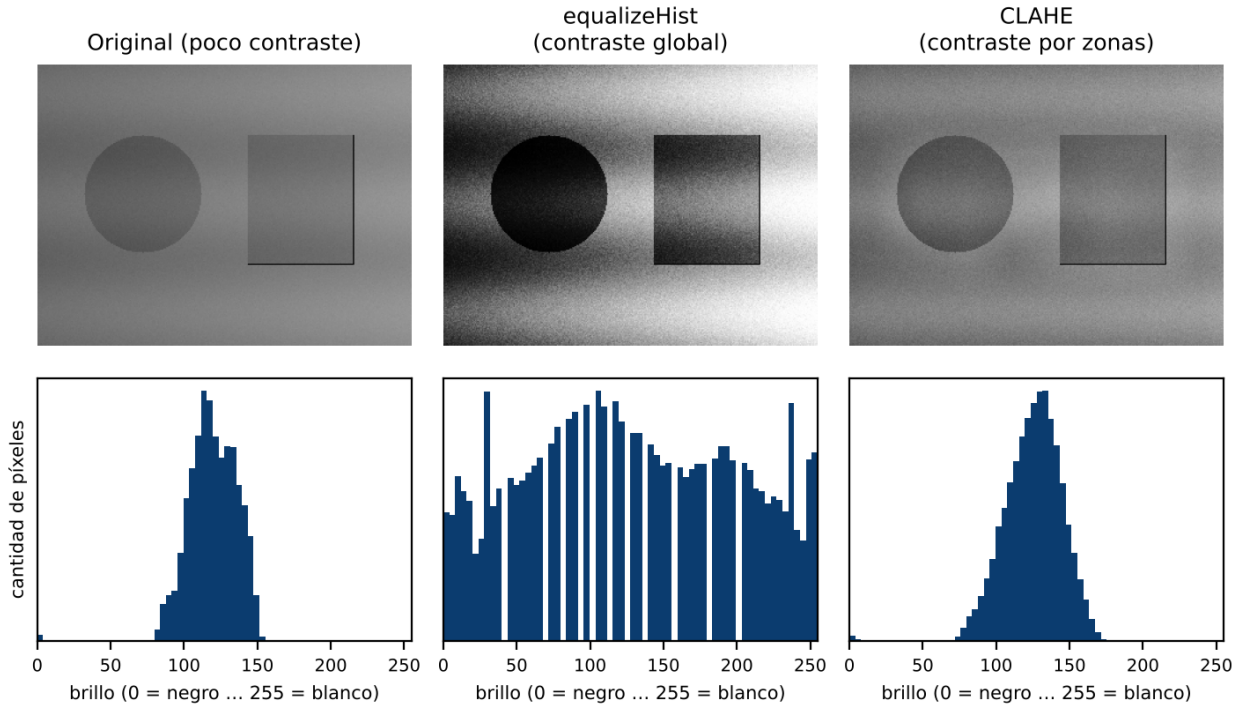


Figura 5. Una imagen con poco contraste y su histograma antes y después. Se usa equalizeHist antes de buscar rostros y CLAHE antes de leer el DPI.

3.6 OCR: convertir una imagen de texto en texto

OCR significa **reconocimiento óptico de caracteres**. El motor usado es **Tesseract**, un programa de código abierto que se instala aparte y que `pytesseract` controla desde Python. Recibe una imagen y devuelve el texto que cree ver.

Ajuste en el código	Para qué sirve
`config="--psm 6"`	*Page segmentation mode* 6: le dice a Tesseract que la imagen es un bloque de texto uniforme (varias líneas seguidas), como la etiqueta.
`--psm 4`	Una sola columna de texto con líneas de distinto tamaño (se prueba también en el DPI).
`--psm 7 / 11`	Una sola línea de texto / texto disperso. Se usan como alternativas cuando el primer intento no sirve.
`tessedit_char_whitelist=ABC…XYZ0123456789<`	Lista de caracteres permitidos. En la zona de lectura mecánica del DPI solo existen letras mayúsculas, números y el signo <, y limitar las opciones evita errores.
`cv2.copyMakeBorder(..., value=255)`	Agrega un marco blanco alrededor de la imagen. Tesseract lee mejor cuando el texto no toca el borde.

Tesseract se equivoca a veces: confunde la letra O con el cero, la I con el uno o pierde las tildes (en el ejemplo, `María` sale como `Maria`). El programa lo asume: compara nombres sin tildes, tolera pequeñas diferencias y, para los números del DPI, corrige las confusiones típicas con una tabla `O→0, I→1, B→8, S→5`.

3.7 Detección de rostros con cascadas de Haar

Para decidir cuál es la foto de medio cuerpo, el programa necesita saber si hay una cara y qué tan grande se ve. Usa un método clásico llamado **detector en cascada de Haar** (Viola y Jones, 2001), que viene incluido en OpenCV. La idea tiene cuatro partes:

- **Características de Haar.** Son rectángulos blancos y negros que miden la diferencia de brillo entre dos zonas. Una cara tiene un patrón de claros y oscuros muy regular: por ejemplo, la franja de los ojos es más oscura que las mejillas.
- **Una ventana que recorre la foto.** El detector desliza una ventana por toda la imagen y la va agrandando (`scaleFactor=1.1` significa un 10 % más cada vez) para encontrar caras de cualquier tamaño.
- **Una cascada de filtros.** En cada posición, la ventana pasa por una serie de pruebas, de la más barata a la más exigente. Si una prueba falla, la posición se descarta enseguida. Esto lo hace lo bastante rápido para correr en una computadora común.
- **Un modelo entrenado.** Qué características usar y con qué valores se aprendió de miles de fotos de caras y de no-caras, y quedó guardado en un archivo `haarcascade_frontalface_default.xml`.

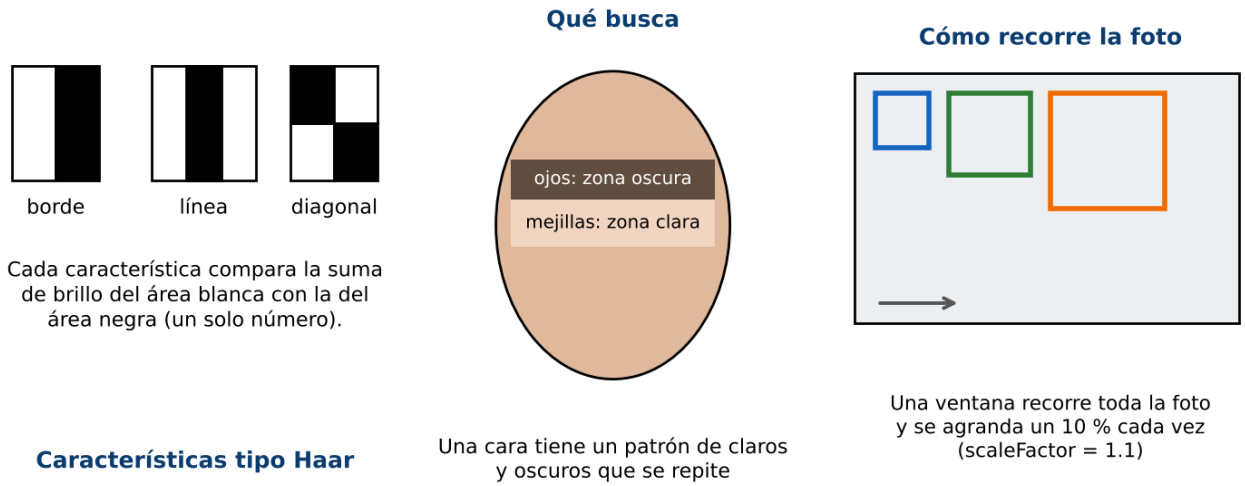


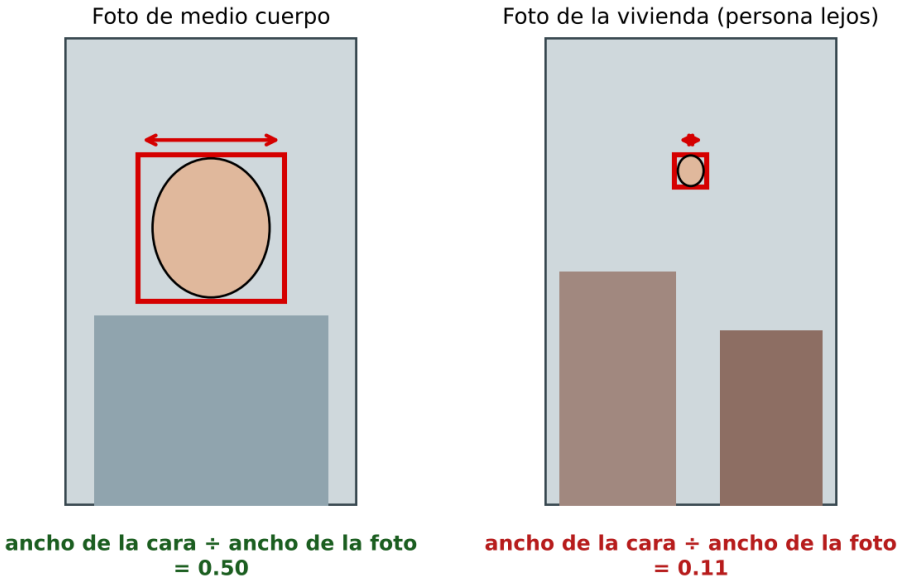
Figura 6. Las tres ideas del detector: características Haar, el patrón de una cara y la ventana que recorre la imagen.

```
caras = _CASCADA.detectMultiScale(gris, scaleFactor=1.1, minNeighbors=5, minSize=(40, 40))
# devuelve una lista de recuadros (x, y, ancho, alto), uno por cada cara encontrada
```

Parámetro	Qué controla	Efecto de subirlo
`scaleFactor`	Cuánto se agranda la ventana en cada paso	Más rápido, pero puede pasar de largo caras de tamaño intermedio
`minNeighbors`	Cuántos recuadros vecinos deben coincidir para aceptar una cara	Menos falsos positivos (paredes, cubetas), pero se pierden caras difíciles
`minSize`	Tamaño mínimo de cara que interesa	Ignora caras pequeñas del fondo

Validar para no equivocarse. Una cara detectada solo se acepta si algo más la confirma: o se encuentra al menos un ojo dentro de ella (`haarcascade_eye.xml`), o **otro modelo distinto** (`frontalface_alt` o `alt2`) encuentra casi el mismo recuadro. Para saber si dos recuadros son "casi el mismo" se usa la función `_solapan`: calcula el área en común dividida entre el área total que cubren juntos (la *intersección sobre la unión*, o IoU) y exige que sea al menos 50 %.

La medida que usa el programa. No importa quién es la persona, sino qué tan grande se ve su cara: el programa calcula **ancho de la cara ÷ ancho de la foto** y se queda con la cara más grande. Un retrato tiene la cara grande; en una foto de la vivienda la persona sale lejos.



Mínimo para ser retrato: MIN_CARA_MEDIO_CUERPO = 0.22 (en las fotos de prueba: retratos 0.43–0.65, viviendas ≤ 0.19)

Figura 7. La proporción de la cara decide quién es la foto de medio cuerpo.

El corte es `MIN_CARA_MEDIO_CUERPO = 0.22`. Se eligió mirando las fotos de prueba: los retratos daban entre 0.43 y 0.65, y las fotos de vivienda no pasaron de 0.19. Si un grupo de fotos no tiene ninguna cara que llegue a ese mínimo (por ejemplo, un primer plano con sombrero), se hace una **segunda pasada más permisiva** (`proporcion_cara_laxa`): prueba varios tamaños de imagen y los tres modelos, sin exigir los ojos.

3.8 Girar la imagen: saber si el DPI está derecho

Las fotos de los DPI llegan acostadas, de cabeza o en cualquier orientación. El programa gira la imagen (`cv2.rotate`), pasa el OCR y cuenta cuántas palabras típicas del DPI aparecen (`GUATEMALA`, `REPUBLICA`, `DOCUMENTO`, `IDENTIFICACION`, `RENAP`…) y cuántos signos `<<`. La orientación con el puntaje más alto es la correcta. Si una orientación ya supera 4 puntos, no se prueban las demás, para ahorrar tiempo.

3.9 La zona de lectura mecánica y el dígito de control

El reverso del DPI trae unas líneas con letras, números y muchos signos `<` (la zona de lectura mecánica, MRZ). Empieza con `IDGTM` y contiene el número de identificación (CUI) y los apellidos y nombres. Tiene una ventaja enorme: incluye un **dígito de control**, un número calculado a partir de los demás que permite saber si el OCR leyó bien o no. El cálculo es el del estándar internacional ICAO, con pesos 7-3-1 que se repiten:

Paso	Ejemplo con los dígitos 1 2 3 4 5 6 7 8 9
1. Multiplicar cada dígito por 7, 3, 1, 7, 3, 1…	1×7=7 · 2×3=6 · 3×1=3 · 4×7=28 · 5×3=15 · 6×1=6 · 7×7=49 · 8×3=24 · 9×1=9
2. Sumar todo	7 + 6 + 3 + 28 + 15 + 6 + 49 + 24 + 9 = 147
3. Quedarse con el último dígito de la suma	147 → **7** es el dígito de control
4. Comparar con el que trae el documento	Si el documento trae 7, la lectura es válida. Si trae otro, el OCR se equivocó en algún dígito.

En el programa, esto es la función `digito_control` (líneas 369 a 375) y se usa en `leer_mrz`. Cuando el CUI del reverso tiene el control correcto, el programa lo marca como **válido** y confía mucho más en él.

3.10 Expresiones regulares: buscar patrones en el texto

Una **expresión regular** es una forma compacta de describir un patrón de texto. El programa define tres:

Patrón	Qué busca	Cómo se lee
`RE_NOTA`	La línea “Nota: …” aunque el OCR la deforme	`(?i)` ignora mayúsculas · `\bn[o0a][tl][aáe4d]?` “Nota” tolerando errores (o/0/a, t/l) · `\s*[:;.,]\s*` los dos puntos o algo parecido · `(.*)` captura el resto de la línea
`RE_TIEMPO`	La fecha y hora, como “10-14-2026 09:41”	`(\d{2})-(\d{2})-(\d{4}) (\d{1,2}):(\d{2})`: mes, día, año, hora y minutos; `\d{2}` es “exactamente dos dígitos”
`RE_CUI`	El CUI impreso: 13 dígitos en grupos de 4, 5 y 4	`(\d{4})\s*(\d{5})\s*(\d{4})` con espacios opcionales entre grupos; `(?<!\d)` y `(?!\d)` exigen que no haya más dígitos pegados

4. Recorrido por el código, función por función

Cada subsección explica una función con tres datos: qué recibe, qué devuelve y la idea clave. Los números de línea son los de la versión entregada del archivo.

4.1 La configuración (líneas 60 a 108)

Constante	Valor	Para qué sirve
`OVERLAY_ALTO / OVERLAY_ANCHO`	0.14 / 0.55	Zona de la etiqueta que se recorta para leer
`UMBRALES`	90 a 140	Los seis niveles de blanco y negro que se prueban
`MIN_CARA_MEDIO_CUERPO`	0.22	Cara mínima (proporción del ancho) para ser retrato
`PARECIDO_NOMBRES`	0.88	Qué tan parecidos deben ser dos nombres para ser la misma persona (0 a 1)
`MIN_LARGO_NUMERO`	8	Dígitos mínimos para considerar que la nota trae un número
`PARTICULAS`	de, del, la, las, los, y	Palabras que no cuentan como nombre (Santos *De La* Cruz)
`PALABRAS_DIRECCION`	barrio, aldea, caserío…	Cuando aparece una, el nombre termina y empieza la dirección
`MIN_VOTOS`	0.34	Fracción mínima de umbrales que deben coincidir para confiar en un nombre
`MAX_VIVIENDA`	3	Fotos de vivienda “normales” (_2, _3, _4); las demás se numeran _5, _6…

Al final de la configuración (líneas 102 a 108) el programa comprueba que OpenCV sea la versión 4 y carga los modelos de rostro desde la carpeta `cv2.data.haarcascades`. Si el programa se bloquea al inicio con un mensaje sobre `CascadeClassifier`, es porque se instaló OpenCV 5.

4.2 Funciones para trabajar con nombres (líneas 120 a 207)

Función	Qué hace	Ejemplo
`normalizar_nombre`	Minúsculas, sin tildes y sin espacios (el nombre que irá en el archivo)	“María López” → `marialopez`
`limpiar_visible`	Deja solo letras, quita palabras sueltas de ruido y corta donde empieza la dirección	“Maria Lopez Barrio El Centro” → “Maria Lopez”
`clave_nombre`	Las dos primeras palabras que no son partículas, sin tildes	“Santos De La Cruz” → `santoscruz`
`mismo_nombre`	¿Son la misma persona? Compara solo esas dos palabras con `difflib.SequenceMatcher` y exige un parecido de 0.88	“Maria Lopez Perez” y “Maria Lopez Perz” → verdadero
`nombre_consenso`	Elige el nombre final: conserva siempre las dos primeras palabras; una tercera solo si otra foto del grupo la confirma	evita que se cuele ruido como “Asce Siok”
`separar_numero_y_nombre`	La nota a veces trae un número y el nombre pasa a la línea siguiente: separa ambos	“Nota: 2468013570101” + línea de abajo

¿Por qué comparar solo las dos primeras palabras? Porque el apellido largo del final es lo que más se corta o se ensucia en el OCR, mientras que el nombre y el primer apellido casi siempre salen bien. La función `difflib.SequenceMatcher(...).ratio()` devuelve un número entre 0 y 1: 1 es idéntico y 0 es

totalmente distinto.

4.3 ocr_overlay (líneas 217 a 281): leer la etiqueta

Recibe la imagen. **Devuelve** cuatro datos: el nombre, la fracción de votos, la hora y el número de la nota. Es la función más delicada del programa y se divide en cinco pasos:

Paso	Qué hace	Líneas
1	Recorta la zona de la etiqueta, la pasa a gris y la agranda 3 veces	227 a 230
2	**Ciclo por cada umbral:** blanco y negro, marco blanco, OCR con `--psm 6` y división del texto en líneas	233 a 240
3	**Ciclo por cada línea:** busca la hora con `RE_TIEMPO` y la nota con `RE_NOTA`; si la nota trae un número largo, lo guarda y toma el nombre de donde corresponda; corta el nombre en la primera coma y limpia el resto	241 a 261
4	Escoge la hora y el número más repetidos entre los seis intentos (`Counter(...).most_common(1)`)	263 a 264
5	**Votación de nombres:** junta las lecturas parecidas en grupos (el ciclo for-else), toma el grupo más grande y de ahí la lectura más repetida (en empate, la más larga). Devuelve `len(mejor) / len(UMBRALES)` como confianza	268 a 281

> **Lo que muestra la Figura 4**
> En el ejemplo, la función devolvió el nombre correcto con confianza 0.5 (tres de seis umbrales), la hora 09:41 y el número de la nota. Además, el nombre salió con una tilde y sin ella según el umbral: el programa lo trata como el mismo porque compara sin tildes.

4.4 proporcion_cara y proporcion_cara_laxa (líneas 303 a 353): medir la cara

Recibe la imagen. **Devuelve** un número de 0 a 1 (0 si no hay cara). Los pasos de `proporcion_cara`:

- Reduce la foto a 900 píxeles de ancho (más rápido y más estable) y la pasa a gris con `equalizeHist`.
- Descarta la franja de la etiqueta (el último 14 %), porque las letras pueden parecer una cara.
- Busca caras con el modelo principal y con dos modelos alternativos.
- **Ciclo por cada cara candidata:** mira si contiene un ojo o si la confirma otro modelo (`_solapan`). Si no, la descarta.
- Devuelve el ancho de la cara válida más grande dividido entre el ancho de la foto.

La versión `proporcion_cara_laxa` solo se usa cuando un grupo no tiene ningún retrato claro: recorre dos tamaños (400 y 700 píxeles), usa los tres modelos con criterios más permisivos y no pide ojos.

4.5 Lectura del DPI (líneas 361 a 523)

Una imagen sin etiqueta se trata como posible DPI. La función `leer_dpi` coordina a las demás en este orden:

Función	Qué hace
`_preparar`	Ajusta la imagen a 1600 píxeles de ancho, la pasa a gris y aplica CLAHE (contraste adaptativo)
`orientar`	Prueba giros y se queda con el que muestra más palabras típicas del DPI (`_puntaje_orientacion`)
`leer_mrz`	Mira desde el 58 % del alto hacia abajo, con seis combinaciones de zona y modo de lectura, y solo caracteres válidos. Busca `GTM`, toma los dígitos siguientes, corrige las confusiones comunes y valida el dígito de control. También extrae apellidos y nombres de la línea con <<

Función	Qué hace
`_parece_frente`	Mira el 40 % de arriba y comprueba que haya al menos dos palabras del encabezado del DPI (“REPUBLICA”, “GUATEMALA”, “DOCUMENTO”…)
`leer_frente`	Lee el CUI impreso en una zona fija (con y sin blanco y negro) y, si hace falta, en toda la imagen; luego las palabras del nombre
`leer_dpi`	Si leyó la zona <<< → es un **reverso**. Si no, pero parece frente y se leyó el CUI o el nombre → es un **frente**. Si no, no se reconoce

4.6 analizar (líneas 529 a 564): el ciclo principal

Recibe la carpeta. **Devuelve** la lista de fichas, una por imagen. Es el primer ciclo de la sección 2.3, con un árbol de decisión por cada imagen:

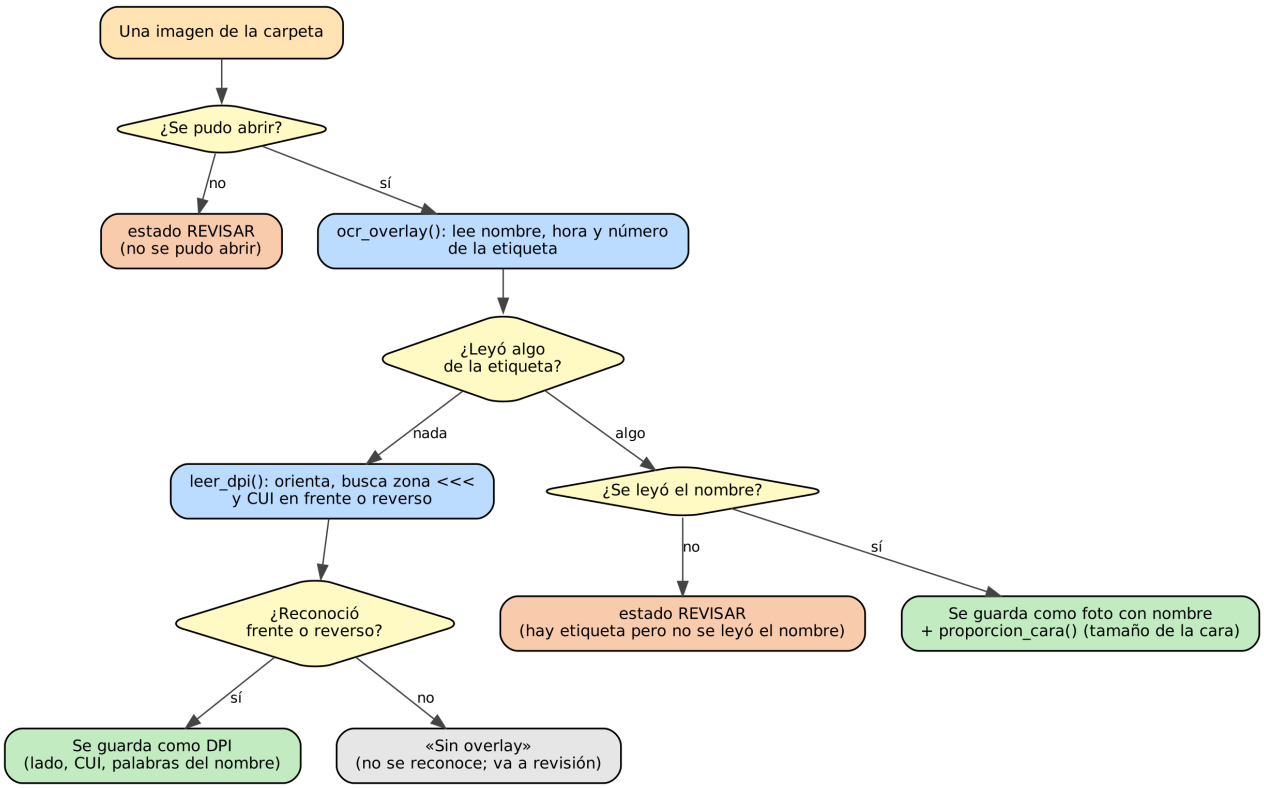


Figura 8. Lo que decide analizar() con cada imagen. Los rombos son preguntas.

4.7 agrupar_personas (líneas 567 a 594): quién es quién

Recibe las fichas. **Devuelve** una lista de grupos, uno por persona, y fija en cada ficha los campos `persona` y `nombre_final`.

- Ordena las fichas por votos de mayor a menor, para que las lecturas más confiables formen los grupos.
- Para cada ficha recorre los grupos existentes: si su nombre `mismo_nombre` algún miembro, entra a ese grupo; si no, abre un grupo nuevo (el ciclo for-else).
- **Regla de seguridad:** si la ficha y el grupo traen números de nota **distintos**, son dos personas distintas aunque el nombre se parezca. Esto separa a familiares con los mismos apellidos.
- Al final, cada grupo elige su nombre final con `nombre_consenso` y la clave con `normalizar_nombre`. La clave es el nombre que llevarán los archivos.

4.8 asignar_roles (líneas 597 a 679): quién es cada foto

Para cada persona (un ciclo por grupo) la función hace lo siguiente:

Paso	Regla
1	Si ninguna cara llega a 0.22, repite la medición con `proporcion_cara_laxa` sobre las fotos del grupo
2	La foto con la cara más grande, si llega al mínimo, es la **de medio cuerpo** y se llama `nombre_1`
3	Las demás se ordenan por la hora de la etiqueta y se numeran desde _2. Hasta _4 son vivienda; de _5 en adelante son "vivienda extra" (el programa avisa, pero no es un error)
4	Si no hubo retrato claro, se propone uno **provisional** (el de cara más grande, o el más temprano si hay empate) y todo el grupo pasa a revisión, porque sin _1 la macro de Word ignora a la persona
5	Fija la **hora del grupo**: la foto más temprana. Es la fecha que se aplica a `_1.jpg` para que Word ordene a las personas en el orden en que se hicieron las visitas
6	Calcula el estado de cada foto: `OK` o `REVISAR` (ver la sección 6)

4.9 emparejar_dpi (líneas 685 a 843): de quién es cada DPI

Es la parte más larga del programa y funciona en tres fases.

Fase 1. Unir cada reverso con su frente por el CUI. Se arma la lista de todas las combinaciones reverso-frente y se mide cuántos dígitos del CUI difieren (`_dif_digitos`). Se ordenan de la diferencia menor a la mayor y se van uniendo: primero las que son idénticas, luego las que difieren en 1 y en 2 dígitos. Esto evita que un frente con un CUI casi igual al de otro DPI (por una lectura imperfecta) se lleve el reverso equivocado. Si un reverso queda igual de cerca de dos frentes, es ambiguo y no se une. Los reversos que quedan sin pareja se unen por apellido solo si a alguno de los dos no se le leyó el CUI.

Fase 2. Asignar cada pareja a una persona. Para cada pareja, y para cada persona del lote, se calcula un puntaje:

Evidencia	Qué pasa	Puntaje
El CUI del DPI coincide con el número de la nota (diferencia d ≤ 2 dígitos)	Es la evidencia más fuerte	`100 × (3 − d)`: 300, 200 o 100
El CUI es distinto del número de la nota de esa persona	**Se descarta a esa persona**, aunque compartan apellidos (familiares)	excluida
Palabras del nombre que coinciden con el DPI	Una sola palabra nunca basta; con dos o más se acepta	+1 por palabra
El nombre de pila coincide	Ayuda a desempatar entre hermanas o familiares	+0.5

Gana la persona con el mejor puntaje. La asignación es **segura** (`alta`) solo si hay un ganador sin empate, no es la coincidencia débil de solo el nombre de pila, y se apoya en el CUI o en un DPI completo (frente y reverso unidos, o CUI válido). Si no, queda en revisión con un mensaje del motivo.

Fase 3. Resolver choques. Si dos parejas quieren a la misma persona, gana la que coincide por CUI. Las demás quedan como *DPI sin asignar*: normalmente son DPI de un familiar o de alguien que no tiene fotos en el lote. Reciben nombres como `DPI_sin_asignar_1_frente` y van a revisión.

4.10 Guardar los resultados (líneas 846 a 1042)

Función	Qué hace
`guardar_como_jpg`	Copia el archivo como .jpg. Un .jpeg es el mismo formato con otra extensión, así que se copia tal cual; un .png se convierte; un DPI girado se endereza. **El original no se toca nunca**
`copiar`	Crea `_salida` y `Revision_manual`. Las fotos en estado OK van a la primera y las demás a la segunda. Si un nombre se repite, agrega `__dup_` para no sobrescribir. Fija la fecha de modificación al momento de la visita con `os.utime`, porque la macro ordena por esa fecha
`escribir_excel`	Crea `registro_fotos.xlsx` con dos hojas: **Registro** (una fila por imagen, con colores: rojo = revisar, amarillo = advertencia, gris = sin reconocer) y **Por persona** (un resumen por persona). Si el Excel está abierto y no se puede escribir, lo guarda con otro nombre con hora
`main`	Lee los argumentos de la línea de comandos (`argparse`), revisa que `_salida` no tenga fotos de otra corrida y llama en orden a analizar, agrupar, asignar roles, emparejar DPI, copiar y Excel. Imprime el resumen y escribe `resumen.json`, que es lo que UiPath puede leer

5. Un ejemplo completo, paso a paso

Datos inventados: una carpeta con seis imágenes de una misma persona, “María López Pérez”, con número de nota **2468013570101**.

Imagen	Qué es	Qué lee analizar()
IMG_A	Retrato de la señora	Nombre «Maria Lopez Perez», 4 de 6 votos, hora 09:41, número 2468013570101, cara 0.52
IMG_B	Fachada de la casa	Mismo nombre y número, hora 09:43, cara 0.00 (no se ve a la persona)
IMG_C	Cocina	Mismo nombre y número, hora 09:44, cara 0.00
IMG_D	Dormitorio	Mismo nombre y número, hora 09:46, cara 0.00
IMG_E	DPI frente (sin etiqueta)	No se leyó etiqueta → `leer_dpi`: lado frente, CUI leído 2468013570107 (un dígito mal), palabras LOPEZ PEREZ
IMG_F	DPI reverso (sin etiqueta)	Zona <<< leída: CUI 2468013570101 con dígito de control válido, palabras LOPEZ PEREZ MARIA

Etapa	Qué pasa	Resultado
agrupar_personas	Las cuatro fotos con etiqueta tienen nombres parecidos y el mismo número: forman un solo grupo	Clave `marialopezperez`
asignar_roles	La cara 0.52 supera 0.22: IMG_A es el retrato. Las otras tres se ordenan por hora	IMG_A=_1; IMG_B=_2; IMG_C=_3; IMG_D=_4. Hora del grupo: 09:41
emparejar_dpi, fase 1	El CUI del reverso y el del frente difieren en 1 dígito (≤ 2) y no hay ambigüedad	Frente y reverso quedan unidos
emparejar_dpi, fase 2	El CUI del reverso coincide con el número de la nota (d = 0): 100 × (3 − 0) = 300, más 3 palabras del nombre	Asignación segura (alta)
copiar	Se copian las seis imágenes con su nombre nuevo y se fija la fecha 09:41 al archivo _1	Todo en `_salida`

```
_salida\
    marialopezperez_1.jpg        (retrato, fecha 09:41)
    marialopezperez_2.jpg        (fachada)
    marialopezperez_3.jpg        (cocina)
    marialopezperez_4.jpg        (dormitorio)
    marialopezperez_DPI1.jpg     (frente)
    marialopezperez_DPI2.jpg     (reverso)
```

Estos seis archivos son justo los que la macro de Word necesita para armar las tres páginas de la persona: vivienda, retrato y DPI.

6. Cuándo algo va a Revision_manual

El principio de diseño es **no adivinar**: si la evidencia no es clara, el archivo se copia a `Revision_manual` y queda anotado el motivo en el Excel. Estos son todos los casos, tal como los define el código:

Caso	Cómo queda
No se pudo abrir la imagen	Estado REVISAR; se conserva el nombre original
Hay etiqueta (se leyó la hora o un número) pero no se pudo leer el nombre	REVISAR; se conserva el nombre original

Caso	Cómo queda
Imagen sin etiqueta que no se reconoce como DPI	“Sin overlay”; se conserva el nombre original
La persona no tiene ninguna cara que llegue al mínimo (retrato provisional)	Todo el grupo en REVISAR: sin _1 la macro ignora a la persona
Nombre leído con baja confianza (menos de 0.34) y sin otra foto que lo respalde	REVISAR
DPI: persona probable pero evidencia insuficiente, o empate entre varias personas	REVISAR con el nombre propuesto
DPI: dos DPI quieren a la misma persona y ninguno gana por CUI	REVISAR
DPI que no coincide con nadie del lote (familiar o persona sin fotos)	REVISAR como `DPI_sin_asignar_N_lado`
DPI con orientación dudosa	REVISAR

Hay casos que **no** son errores y se quedan como OK, con una advertencia amarilla en el Excel: la persona tiene menos de tres fotos de vivienda, tiene fotos adicionales (de la quinta en adelante) o le falta el frente o el reverso del DPI.

7. Cómo explicarlo en la presentación

7.1 La explicación en tres minutos

Minuto	Qué decir
0:00	“Cada semana llegan entre 486 y 600 fotos por WhatsApp, con nombres automáticos y mezcladas. Para armar el expediente, alguien tiene que saber de quién es cada una y renombrarla a mano. Nuestro programa lo hace solo.”
0:40	“Funciona como lo haría una persona: lee la etiqueta que trae cada foto, junta las de la misma persona, busca la que tiene la cara más grande para el retrato, ordena las demás y empareja su DPI.”
1:20	“Para leer la etiqueta usamos OCR, un lector de texto instalado en la misma computadora. Como una sola lectura puede fallar, la hacemos seis veces con distinto contraste y votamos. Para el retrato usamos un detector de rostros clásico de OpenCV que solo mide qué tan grande es la cara; no identifica a nadie.”
2:00	“Al DPI lo reconocemos por la zona de lectura mecánica del reverso, que trae el CUI con un dígito de control. Ese dígito nos dice si el OCR leyó bien. Con el CUI unimos frente y reverso y lo asignamos a la persona.”
2:30	“Todo es local: ninguna imagen sale de la computadora. Y si algo no es seguro, no lo adivinamos: va a una carpeta de revisión manual y queda anotado en un Excel.”

7.2 Preguntas que pueden hacerte

Pregunta	Respuesta sugerida
¿Usa inteligencia artificial?	Usa dos herramientas de aprendizaje automático ya entrenadas que corren localmente: un detector de rostros clásico y el OCR Tesseract. El resto es lógica de programación. No usa IA generativa ni servicios en la nube.
¿Las fotos se envían a internet?	No. Todo se procesa en la computadora del usuario. Es importante porque son datos personales de adultos mayores.
¿Reconoce a las personas por la cara?	No. Solo detecta si hay una cara y mide su tamaño para elegir la foto de medio cuerpo. La identidad sale del texto de la etiqueta y del DPI.
¿Cómo sabe cuál es la foto de medio cuerpo?	Es la foto de la persona con la cara más grande, si supera 22 % del ancho de la foto. Las demás son de la vivienda.
¿Cómo sabe de quién es un DPI?	Por el CUI: lo compara con el número que trae la nota de las fotos y valida el dígito de control del reverso. Si no hay CUI, usa el nombre, pero con reglas más estrictas.
¿Qué pasa si se equivoca?	Está diseñado para equivocarse lo menos posible y avisar. Lo dudoso va a Revision_manual. En las pruebas, el lote ordenado salió sin casos en revisión y el desordenado con cerca de 5 %.
¿Por qué no usar una IA más potente?	Por privacidad (los datos no deben salir), porque funciona sin internet y sin costo, y porque para esta tarea las técnicas clásicas alcanzan. Podría explorarse más adelante en un entorno controlado.
¿Cuánto tarda?	Cerca de cinco minutos con 90 imágenes en nuestras pruebas. El tiempo se va sobre todo en el OCR, que se repite seis veces por foto.
¿Qué limitaciones tiene?	Las fotos sin etiqueta y los reversos ilegibles no se pueden asignar solos. Depende de que la etiqueta de la cámara esté presente y legible.

7.3 Lo que conviene decir y lo que no

Sí decir	Evitar
“Detecta rostros para elegir el retrato”	“Reconoce a las personas”
“Procesa todo localmente”	“Usa inteligencia artificial en la nube”
“Lee la etiqueta con OCR y vota entre seis lecturas”	“Lee la etiqueta perfectamente”
“Lo dudoso va a revisión manual”	“No se equivoca”
“Reduce el trabajo de renombrar y buscar fotos”	Cifras de ahorro de tiempo que no hemos medido

7.4 Limitaciones y una mejora pendiente

Al preparar este documento, probando la función de lectura de la etiqueta con una imagen de ejemplo, apareció un detalle real que puedes mencionar como mejora futura: cuando la nota trae un número y el nombre pasa a la línea siguiente, la función `separar_numero_y_nombre` toma **exactamente la línea siguiente**. Si Tesseract deja una línea en blanco en medio, en ese umbral el nombre se pierde. Hoy la votación lo compensa (en el ejemplo, tres de los seis umbrales alcanzaron), pero la confianza baja y el nombre depende de menos lecturas. La mejora sería saltar las líneas vacías y tomar la primera con texto. No la apliqué en el archivo entregado para no cambiar una versión ya probada.

8. Cómo seguir aprendiendo

8.1 En qué orden leer el archivo

- Empieza por `main()` (línea 980): es corta y muestra el orden de todo.
- Sigue con `analizar()` (529): el ciclo principal y el árbol de decisión.
- Después `ocr_overlay()` (217): la lectura de la etiqueta y la votación.
- Luego `asignar_roles()` (597) y `agrupar_personas()` (567).
- Deja para el final la lectura del DPI (361 a 523) y `emparejar_dpi()` (705): son las partes más largas.

8.2 Experimentos seguros

Antes de tocar nada, **haz una copia del archivo**. Así, si algo se rompe, vuelves a la versión que funciona.

Experimento	Qué deberías observar
Ejecutar con `--solo-analizar` y abrir `registro_fotos.xlsx`	El programa no copia nada; el Excel muestra cómo leyó cada imagen (nombre, votos, tamaño de cara, rol, estado)
Cambiar `MIN_CARA_MEDIO_CUERPO` de 0.22 a 0.45	Menos fotos califican como retrato; aparecen grupos con retrato provisional y más archivos en revisión
Cambiar `UMBRALES` a `(120,)`	La corrida es unas seis veces más rápida en la lectura de la etiqueta, pero los votos son 0 o 1 y hay más lecturas fallidas
Agregar `print(nombres)` dentro de `ocr_overlay` antes de la votación	Ver las seis lecturas del nombre y cuáles coinciden
Cambiar `PARECIDO_NOMBRES` a 0.95	El agrupamiento es más estricto: nombres con pequeños errores se separan en personas distintas

8.3 Glosario

Término	Significado
Píxel	Cada punto de una imagen digital; un número (gris) o tres (color)
Canal / BGR	Cada color base de un píxel; OpenCV usa el orden azul-verde-rojo
Escala de grises	Imagen con un solo número por píxel, de 0 (negro) a 255 (blanco)
Umbral / binarización	Número de corte que convierte cada píxel en blanco o negro
Histograma / ecualización	Conteo de píxeles por brillo / estiramiento del brillo para ganar contraste
OCR	Reconocimiento óptico de caracteres: de imagen de texto a texto
Cascada de Haar	Detector de objetos (aquí, rostros) hecho de filtros encadenados, entrenado con ejemplos
Falso positivo / negativo	Detectar algo que no está (una pared como cara) / no detectar algo que sí está
IoU	Intersección sobre unión: cuánto se solapan dos recuadros, de 0 a 1
Expresión regular	Patrón compacto para buscar texto (la nota, la hora, el CUI)
CUI	Código Único de Identificación del DPI (13 dígitos)
MRZ / dígito de control	Zona de lectura mecánica del reverso del DPI / número que verifica que los demás se leyeron bien
Heurística	Regla práctica que funciona casi siempre, aunque no sea una garantía
Diccionario, lista, conjunto	Colecciones de datos de Python: con etiquetas, ordenada, sin repetidos
Función / ciclo	Bloque reutilizable con nombre / repetición de pasos
Fecha de modificación (mtime)	Fecha del archivo que la macro de Word usa para ordenar a las personas

Anexo. Mapa de funciones del archivo

Función	Línea	Función	Línea
`leer_imagen`	114	`digito_control`	369
`normalizar_nombre`	120	`_preparar`	385
`limpiar_visible`	132	`orientar`	401
`clave_nombre / mismo_nombre`	148 / 155	`leer_mrz`	423
`nombre_consenso`	167	`_parece_frente`	458
`separar_numero_y_nombre`	193	`leer_frente`	467
`ocr_overlay`	217	`leer_dpi`	504
`_solapan`	292	`analizar`	529
`proporcion_cara`	303	`agrupar_personas`	567
`proporcion_cara_laxa`	339	`asignar_roles`	597
`emparejar_dpi`	705	`guardar_como_jpg`	846
`copiar`	862	`escribir_excel`	887
`main`	980		